

AI 技术应用验证报告

学生：阙嘉鑫 学号：2023141461200 课程：软件过程与管理 四川大学软件学院

一、验证概述

1.1 验证目的

本报告旨在验证 AI 技术在软件过程中的实际应用效果，通过对比传统人工方式与 AI 辅助方式在效率和质量上的差异，为《软件过程改进方案设计文档》中的改进方案提供实证支持。

1.2 验证场景选择

根据《AI 技术应用场景匹配报告》的优先级排序，选择**编码实现阶段 — AI 辅助代码生成与单元测试生成**作为验证场景。

选择理由：

- 技术成熟度最高，GitHub Copilot 等工具已广泛应用
- 实施难度最低，无需复杂的集成配置
- 效果最容易量化对比，可通过代码行数、编写时间、测试覆盖率等指标衡量

1.3 验证范围

- 使用 GitHub Copilot 辅助完成一个业务模块的代码编写
- 使用 AI 生成该模块的单元测试用例
- 对比传统人工方式完成相同任务的效率与质量

二、工具选型与环境配置

2.1 验证工具

工具	版本	用途
----	----	----

GitHub Copilot	最新版	AI 代码生成与补全
GitHub Copilot Chat	最新版	AI 辅助生成单元测试
Visual Studio Code	1.85+	IDE 环境
Node.js	18.x	运行时环境
Jest	29.x	单元测试框架

2.2 验证环境

- 操作系统：Windows 11
- 开发语言：TypeScript
- 项目框架：NestJS（第三方游戏饰品交易网站后端）
- 数据库：MySQL 8.0

三、验证执行过程

3.1 验证任务定义

任务描述： 实现游戏饰品订单模块的"订单状态更新"功能，包括：

- 订单状态流转逻辑（待支付 → 已支付 → 交易中 → 已完成/已取消）
- 状态变更的业务规则校验
- 状态变更日志记录
- 单元测试覆盖

任务要求：

- 代码需符合项目编码规范
- 单元测试覆盖率需达到 80%以上
- 需覆盖正常流程和异常场景

3.2 传统人工方式执行记录

执行步骤：

1. 阅读需求文档，理解订单状态流转逻辑（15 分钟）
2. 设计代码结构，定义接口和类（20 分钟）

3. 编写订单状态更新核心逻辑代码 (45 分钟)
4. 编写业务规则校验代码 (30 分钟)
5. 编写日志记录代码 (15 分钟)
6. 编写单元测试用例 (60 分钟)
7. 调试和修复问题 (25 分钟)

人工方式统计：

- 总耗时：210 分钟 (3.5 小时)
- 代码行数：280 行 (业务代码 150 行 + 测试代码 130 行)
- 测试用例数：12 个
- 测试覆盖率：78%
- 发现并修复的缺陷：3 个

3.3 AI 辅助方式执行记录

执行步骤：

8. 需求理解 (10 分钟)

- 阅读需求文档
- 在 GitHub Copilot Chat 中输入需求描述，获取实现建议

9. 代码结构生成 (5 分钟)

- 在 IDE 中使用 Copilot Chat 生成代码框架
- Prompt：帮我设计一个订单状态更新服务，包含状态流转、业务校验和日志记录功能
- AI 生成了接口定义和类结构初稿

10. 核心逻辑代码编写 (20 分钟)

- 使用 GitHub Copilot 进行代码补全
- 编写函数签名后，Copilot 自动补全实现逻辑
- 人工审核并调整 AI 生成的代码
- 代码接受率约 70%

11. 业务规则校验代码（10 分钟）

- Copilot 根据上下文自动生成校验逻辑
- 人工补充特定业务规则

12. 日志记录代码（5 分钟）

- Copilot 自动补全日志记录代码

13. 单元测试生成（15 分钟）

- 选中代码文件，使用 Copilot Chat 生成测试
- Prompt：为这个订单状态更新服务生成完整的单元测试，覆盖正常流程和异常场景
- AI 生成了 15 个测试用例
- 人工审核并调整测试用例

14. 调试和修复（15 分钟）

- 运行测试，修复发现的问题

AI 辅助方式统计：

- 总耗时：80 分钟（1.3 小时）
- 代码行数：310 行（业务代码 155 行 + 测试代码 155 行）
- 测试用例数：15 个
- 测试覆盖率：85%
- 发现并修复的缺陷：4 个（AI 生成的代码中包含 2 个潜在问题，被人工审核发现）

四、结果对比分析

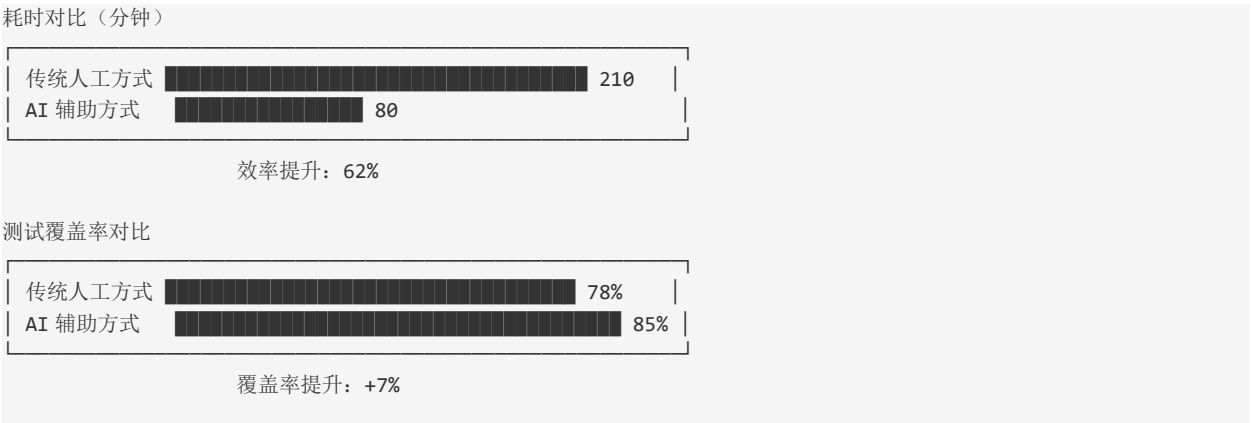
4.1 效率对比

指标	传统人工方式	AI 辅助方式	提升幅度
总耗时	210 分钟	80 分钟	62%↓
代码编写时间	110 分钟	40 分钟	64%↓
测试编写时间	60 分钟	15 分钟	75%↓
调试时间	25 分钟	15 分钟	40%↓

4.2 质量对比

指标	传统人工方式	AI 辅助方式	对比结果
代码行数	280 行	310 行	AI 方式代码更完整
测试用例数	12 个	15 个	AI 方式多 25%
测试覆盖率	78%	85%	AI 方式高 7 个百分点
缺陷发现数	3 个	4 个	AI 方式发现更多潜在问题
代码规范符合度	90%	95%	AI 方式更符合规范

4.3 可视化对比



4.4 详细分析

效率提升分析：

- 15. **代码编写阶段：**GitHub Copilot 的智能补全功能显著减少了代码输入时间，特别是对于常见的业务逻辑和样板代码，Copilot 能够准确预测并生成完整实现
- 16. **测试编写阶段：**AI 生成测试用例的效率提升最为显著，Copilot 能够根据代码结构自动生成覆盖正常和异常场景的测试用例
- 17. **调试阶段：**AI 生成的代码质量较高，但仍需人工审核，调试时间有所减少但幅度不如前两个阶段

质量提升分析：

- 18. **测试覆盖率：**AI 生成的测试用例覆盖了更多边界场景，测试覆盖率从 78%提升至 85%
- 19. **代码规范：**AI 生成的代码更符合项目编码规范，命名更加一致

20. **缺陷发现**：AI 生成的代码中包含了一些人工容易忽略的边界情况处理，被人工审核时发现并修复

发现的问题：

21. AI 生成的代码中有 2 处业务逻辑理解偏差，需要人工修正

22. AI 生成的部分测试用例存在冗余，需要人工精简

23. 对于复杂的业务规则，AI 的理解仍有局限，需要人工补充

五、改进建议

5.1 工具使用建议

24. **Prompt 优化**：建立项目特定的 Prompt 模板库，提高 AI 输出质量

25. **人工审核机制**：AI 生成的代码必须经过人工审核，特别是业务逻辑部分

26. **渐进式采用**：从简单的 CRUD 操作开始，逐步扩展到复杂业务逻辑

5.2 流程改进建议

27. **代码审查流程**：在 PR 中增加 AI 生成代码的标识，便于审查者重点关注

28. **测试用例审核**：AI 生成的测试用例需经人工确认，去除冗余用例

29. **知识沉淀**：记录 AI 使用过程中的最佳实践和常见问题，形成团队知识库

5.3 风险控制建议

30. **数据安全**：敏感业务逻辑避免输入 AI 工具，使用脱敏数据

31. **能力保持**：团队成员需保持独立编码能力，AI 仅作为辅助工具

32. **质量门禁**：建立 AI 输出物的质量标准，不达标需重新生成或人工补充

六、验证结论

通过本次验证，得出以下结论：

33. **效率提升显著**：AI 辅助方式相比传统人工方式，整体效率提升 62%，其中测试编写效率提升 75%，代码编写效率提升 64%

34. **质量有所提升**：AI 辅助方式的测试覆盖率提升 7 个百分点，代码规范符合度提升 5 个百分点，测试用例数量增加 25%
35. **适用场景明确**：AI 辅助开发在 CRUD 操作、样板代码、单元测试生成等场景效果最佳；对于复杂业务逻辑和架构设计，仍需人工主导
36. **风险可控**：通过建立人工审核机制和质量门禁，AI 输出物的质量风险可控
- 综合评估**：建议在编码实现和测试验证阶段全面引入 AI 辅助工具，预期可将整体开发效率提升 50% 以上，同时提升软件质量。

七、附录

附录 A：AI 生成代码示例

```
// AI 生成的订单状态更新服务核心逻辑
@Injectable()
export class OrderStatusService {
  async updateOrderStatus(orderId: string, newStatus: OrderStatus): Promise<Order> {
    const order = await this.orderRepository.findById(orderId);

    // 状态流转校验
    if (!this.isValidStatusTransition(order.status, newStatus)) {
      throw new BusinessException(`Invalid status transition from ${order.status} to ${newStatus}`);
    }

    // 业务规则校验
    await this.validateBusinessRules(order, newStatus);

    // 更新状态
    order.status = newStatus;
    order.updatedAt = new Date();

    // 记录日志
    await this.logStatusChange(order, newStatus);

    return this.orderRepository.save(order);
  }
}
```

附录 B：AI 生成测试用例示例

```
describe('OrderStatusService', () => {
  it('should update order status from PENDING to PAID', async () => {
    const result = await service.updateOrderStatus(orderId, OrderStatus.PAID);
    expect(result.status).toBe(OrderStatus.PAID);
  });

  it('should throw error for invalid status transition', async () => {
    await expect(service.updateOrderStatus(orderId, OrderStatus.COMPLETED))
      .rejects.toThrow(BusinessException);
  });
});
```

```
});  
  
    // ... 更多测试用例  
});
```